Assignment 2

Task 1: Call by Value, Reference and Pointer

1. Aim

To write a C++ program demonstrating call by value, call by reference, and call by pointer, and to understand return by reference and default arguments.

2. C++ Program – Predict the Output

```
None

#include <iostream>
using namespace std;

void change(int a, int& b, int* c)
{
   a += 10;
   b += 10;
   *c += 10;
}

int& bigger(int& p, int& q)
{
   return p > q ? p : q;
}

void show(int x, int y = 5, int z = 10)
{
```

```
    cout << x + y + z << ' ';
}

int main()
{
    int a = 1, b = 2, c = 3;

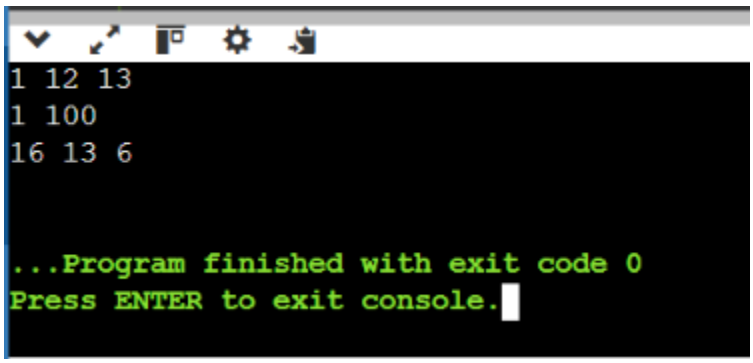
    change(a, b, &c);
    cout << a << ' ' << b << ' ' << c << '\n';

    bigger(a, c) = 100;
    cout << a << ' ' << c << '\n';

    show(1);
    show(1, 2);
    show(1, 2, 3);
    cout << '\n';

    return 0;
}
```

Output



4. Explanation

- a is passed by value. Only the local copy changes, so the original a remains 1.
- b is passed by reference. Its original value changes from 2 to 12.
- c is passed by pointer. Its original value changes from 3 to 13.
- bigger(a,c) returns a reference to the larger variable. Assignment of 100 changes a.
- show(1) uses default arguments y=5, z=10.
- show(1,2) uses default argument z=10.
- show(1,2,3) uses all supplied arguments.

5. Complete the Code – Call by Reference

```
None

#include <iostream>
using namespace std;

void applyByValue(long long fee, int percent)
{
   fee -= fee * percent / 100;
}

void applyConcession(long long& fee, int percent)
{
   fee -= fee * percent / 100;
}

int main()
{
   long long hostelFee = 45000;

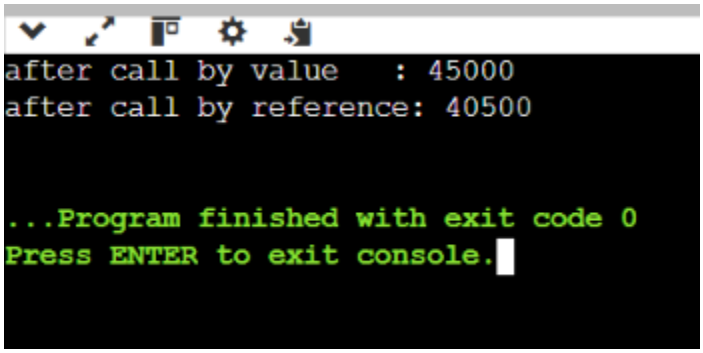
   applyByValue(hostelFee, 10);
```

```
    cout << "after call by value   : " << hostelFee << '\n';

    applyConcession(hostelFee, 10);
    cout << "after call by reference: " << hostelFee << '\n';

    return 0;
}
```

Output



7. Alternative Concept and Complexity Analysis

If the function returns the updated fee instead of modifying it through a reference:

```
None
long long applyConcession(long long fee, int percent)
{
    return fee - fee * percent / 100;
}
```

Calling code:

```
None
hostelFee = applyConcession(hostelFee, 10);
```

Method	**Time Complexity**	**Space**
Call by value	O(1)	Copies parameter
Call by reference	O(1)	No object copy
Call by const reference	O(1)	No object copy

A const reference is used when a function needs to read data without modifying it.

Task 2 – Return by Reference

Aim

To demonstrate return by reference and understand dangling references.

C++ Program

```
None

#include <iostream>
using namespace std;

int& highestMark(int marks[], int n)
{
```

```
    int max = 0;
    for (int i = 1; i < n; i++)
        if (marks[i] > marks[max])
            max = i;
    return marks[max];
}

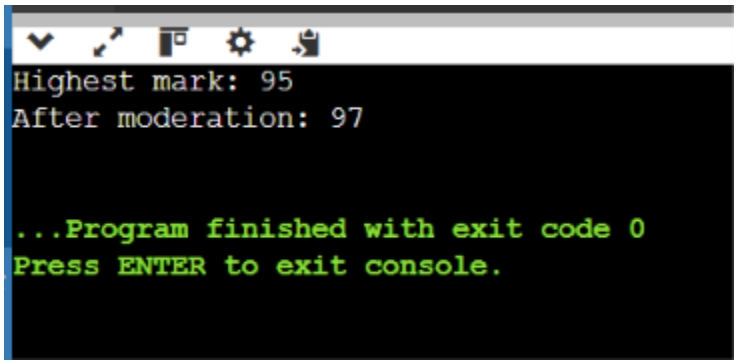
int main()
{
    int marks[5] = {75, 90, 85, 95, 80};

    cout << "Highest mark: " << highestMark(marks, 5) << endl;

    highestMark(marks, 5) += 2;

    cout << "After moderation: " << highestMark(marks, 5) << endl;
    return 0;
}
```

Output



Explanation

- Return by reference allows modifying the original array element.
- highestMark(marks,5) += 2 increases the highest mark.
- Returning a local variable reference causes a dangling reference and undefined behaviour.

Complexity

Method	**Time**	**Space**
Reference	O(n)	O(1)
Index	O(n)	O(1)
Pointer	O(n)	O(1)

Conclusion: Return by reference modifies original data directly, but returning references to local variables is unsafe.

Task 3 : Default Arguments and Function Overloading

Aim

To implement function overloading and default arguments in C++.

C++ Program

```
None

#include <iostream>
using namespace std;
```

```
double charge(double weight, bool express = false,
              bool insured = false)
{
   double cost = weight * 50;

   if (express)
      cost += 100;

   if (insured)
      cost += 50;

   return cost;
}

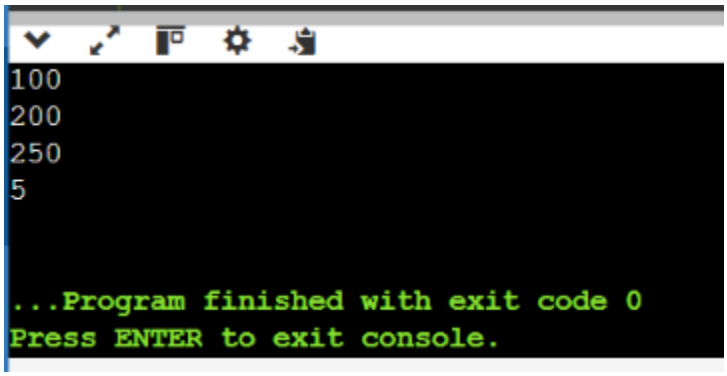
double charge(int grams)
{
   return grams * 0.05;
}

int main()
{
   cout << charge(2) << endl;
   cout << charge(2.0) << endl;
   cout << charge(2.0f) << endl;
   cout << charge(2, true) << endl;
   cout << charge(2, true, true) << endl;
   cout << charge(100) << endl;

   return 0;
```

```
}
```

Output



Explanation

- Function overloading means multiple functions have the same name but different parameters.
- Default arguments allow omitting optional parameters.
- charge(2) selects the int overload.
- charge(2.0) selects the double overload.
- charge(2.0f) converts float to double.
- Default arguments must be specified from right to left.

Complexity

Operation	**Complexity**
Function execution	O(1)

Overload resolution	Compile time
Space	O(1)

Conclusion: Function overloading supports different input types, and default arguments simplify function calls. Avoid ambiguous overloads.

Task 4 : Specifying a Class with an Array Inside It

Aim

To implement a SavingsAccount class using an array to store the last 10 transactions.

C++ Program

```
None

#include <iostream>
using namespace std;

class SavingsAccount
{
   int accountNo;
   double balance;
   double transactions[10];
   int count;

public:
   SavingsAccount(int a, double b)
   {
```

```
        accountNo = a;
        balance = b;
        count = 0;
    }

    void deposit(double amount);
    void withdraw(double amount);
    void miniStatement();
};

void SavingsAccount::deposit(double amount)
{
    balance += amount;
    if (count < 10)
        transactions[count++] = amount;
}

void SavingsAccount::withdraw(double amount)
{
    balance -= amount;
    if (count < 10)
        transactions[count++] = -amount;
}

void SavingsAccount::miniStatement()
{
    cout << "Balance: " << balance << endl;
    cout << "Transactions: ";
    for (int i = 0; i < count; i++)
```

```
        cout << transactions[i] << " ";
    cout << endl;
}

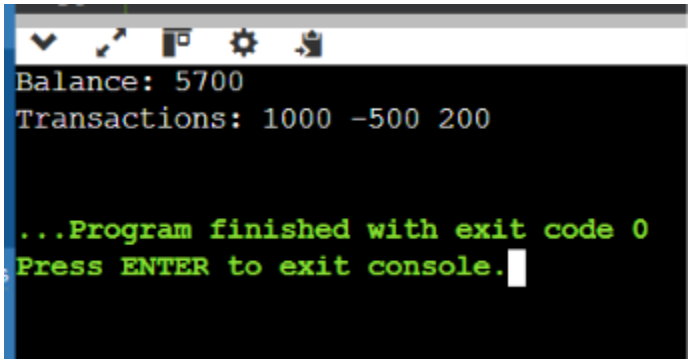
int main()
{
    SavingsAccount s(101, 5000);

    s.deposit(1000);
    s.withdraw(500);
    s.deposit(200);

    s.miniStatement();

    return 0;
}
```

Output



Explanation

- Private members protect account data.
- deposit() adds money.

- withdraw() deducts money.
- miniStatement() displays transactions.
- Member functions can be defined outside the class using ::.

Complexity

Method	**Time**
Add transaction (shifting)	O(k)
Circular buffer insertion	O(1)
Space	O(10)

Conclusion: A circular buffer is better for storing the latest 10 or 50 transactions because insertion takes O(1) time without shifting elements.

Task 5 : Static Members and Arrays of Objects

Aim

To implement static data members and arrays of objects for bus ticket booking.

C++ Program

```
None

#include <iostream>
#include <string>
```

```
using namespace std;

class BusTicket
{
    static int issued;
    static const int CAPACITY = 3;

    int seat;
    string name;

public:
    BusTicket(const string& n) : seat(0), name(n)
    {
        if (issued < CAPACITY)
            seat = ++issued;
    }

    static int count()
    {
        return issued;
    }

    void print() const
    {
        cout << name << ": "
             << (seat ? "Seat " + to_string(seat) : "Waiting")
             << endl;
    }
};
```

```
int BusTicket::issued = 0;

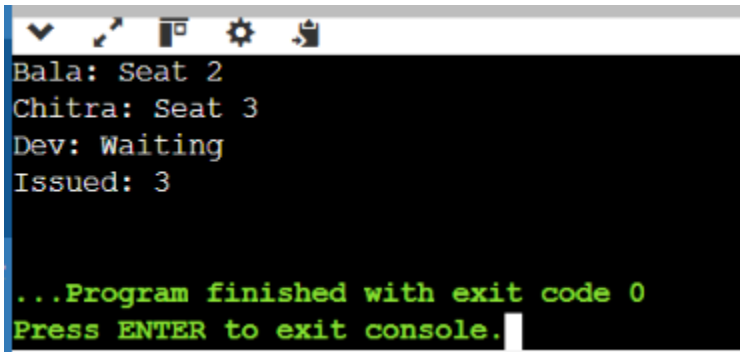
int main()
{
    BusTicket trip[4] = {
        BusTicket("Asha"), BusTicket("Bala"),
        BusTicket("Chitra"), BusTicket("Dev")
    };

    for (const BusTicket& t : trip)
        t.print();

    cout << "Issued: " << BusTicket::count() << endl;

    return 0;
}
```

Output



Explanation

- issued is a static data member shared by all objects.

- CAPACITY represents the maximum seats.
- Static function count() returns issued seats.
- Array trip[4] creates four ticket objects.
- The fourth ticket is placed on the waiting list.

Complexity

Operation	**Complexity**
Ticket creation	O(1)
Counting tickets	O(1)
Space	O(n)

Conclusion: Static members maintain a common seat count for all tickets. They are useful for shared class information.

Task 6 : Friend Functions and Inline Functions

Aim

To implement friend functions for comparing time objects and calculating time differences.

C++ Program

```
None

#include <iostream>
using namespace std;
```

```
class Time
{
   int h, m;

public:
   Time(int hh, int mm) : h(hh), m(mm) {}

   friend int minutesBetween(const Time& a, const Time& b);
   friend bool earlier(const Time& a, const Time& b);
};

inline int minutesBetween(const Time& a, const Time& b)
{
   int t1 = a.h * 60 + a.m;
   int t2 = b.h * 60 + b.m;

   return (t1 > t2) ? t1 - t2 : t2 - t1;
}

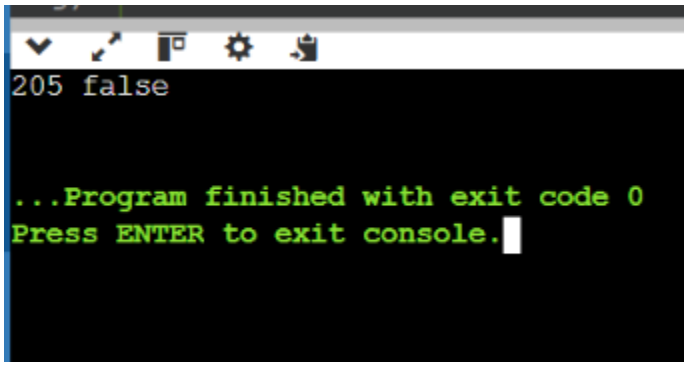
inline bool earlier(const Time& a, const Time& b)
{
   return (a.h * 60 + a.m) < (b.h * 60 + b.m);
}

int main()
{
   Time departure(9, 45), arrival(13, 10);
```

```
    cout << minutesBetween(departure, arrival) << " "
         << boolalpha << earlier(arrival, departure) << endl;

    return 0;
}
```

Output



Explanation

- Friend functions access private members of a class.
- minutesBetween() calculates the difference in minutes.
- earlier() checks which time comes first.
- inline suggests the compiler expand a function at its call site to reduce function-call overhead.

Complexity

Operation	**Complexity**
Time difference	O(1)

Time comparison	O(1)
Space	O(1)

Conclusion: Friend functions provide access to private members, while inline functions may improve performance for small functions.

Task 7 : Diagnose, Redesign and Validate A. Diagnose the Limitation :

The original SavingsAccount program stores transactions using a fixed array. When a new transaction is added after the array becomes full, older transactions must be shifted to make space.

This causes unnecessary data movement and increases insertion time.

B. Define the Design Objective

The objective is to redesign the transaction storage system to:

- Store only the latest 10 transactions.
- Insert new transactions without shifting existing elements.
- Reduce transaction insertion time from O(k) to O(1).
- Use the same fixed memory capacity.

C. Design the Revised Solution

Proposed Design: Circular Buffer

A circular buffer uses an array and a next-position index. When the array becomes full, new transactions overwrite the oldest transactions.

Algorithm

1. Initialize count and next to zero.
2. Insert transaction at next position.
3. Increment next using modulo 10.
4. Increase count until it reaches 10.
5. Display transactions from the oldest to newest.

C++ Program

```
None

#include <iostream>
using namespace std;

class TransactionBuffer
{
    int transactions[10];
    int count = 0;
    int next = 0;

public:
    void add(int amount)
    {
        transactions[next] = amount;
        next = (next + 1) % 10;

        if (count < 10)
            count++;
    }

    void display()
    {
        cout << "Transactions: ";
```

```
        int start = (next - count + 10) % 10;

        for (int i = 0; i < count; i++)
            cout << transactions[(start + i) % 10] << " ";

        cout << endl;
    }
};

int main()
{
    TransactionBuffer t;

    for (int i = 1; i <= 12; i++)
        t.add(i * 100);

    t.display();

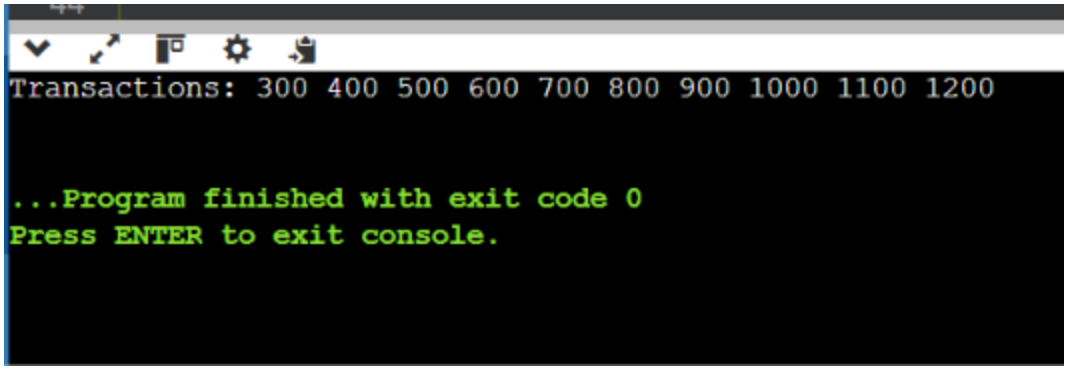
    return 0;
}
```

D. Predict Before Implementing

The circular buffer will store the latest 10 transactions. Insertion will take constant time O(1) without shifting elements.

E. Implement and Validate

Output



The output confirms that the first two transactions (100 and 200) were overwritten by the latest transactions.

F. Defend the Final Design

Feature	**Original Array**	**Circular Buffer**
Insertion time	O(k)	O(1)
Memory	O(10)	O(10)
Shifting	Required	Not required
Storage	Older entries retained	Latest 10 retained

Conclusion

The circular buffer improves transaction insertion efficiency by eliminating shifting. It uses the same memory as the original fixed array and efficiently maintains the latest 10 transactions. Therefore, it is suitable for a bank mini-statement system with frequent transactions.